

Unit 4: Profiles, Resources, and Auditing

Profiles and Resources

- A **Profile** in Oracle is a collection of resource limits and password parameters that can be assigned to database users. Profiles help in enforcing security policies, managing resources, and controlling user behaviour within the database.
- **Resources** in Oracle Database refer to the physical and logical components of the database system that can be managed and controlled. Resources encompass various aspects such as CPU, memory, I/O, and sessions.
- **Profiles and resources** offer a powerful mechanism for managing user access and resource consumption in Oracle databases.

Overview of User Profiles

- A **Profile** in Oracle is a collection of resource limits and password parameters that can be assigned to database users.
- A **user profile** is a collection of resource limits and password-related attributes determining the rules and restrictions for logging in and managing passwords and resources within a database. The database can contain multiple user profiles, each associated with zero to many users. Each user in an Oracle database is assigned to a single user profile at any given time. If a user is not explicitly assigned to a profile, they will be automatically assigned to the DEFAULT profile.

Profile Resource Parameters

- Profile resource parameters are the heart of resource management in Oracle databases. They define the maximum amount of system resources a user assigned to the profile can consume during a session. This allows you to control how users utilize resources like CPU, memory, and temporary space, ensuring optimal usage and preventing any single user from monopolizing resources.
- Here are some common profile resource parameters you can configure:
 - **SESSIONS_PER_USER:** Specify the number of concurrent sessions to which you want to limit the user.

- **CPU_PER_SESSION:** Specify the CPU time limit for a session, expressed in hundredths of seconds.
- **CPU_PER_CALL:** Specify the CPU time limit for a call (a parse, execute, or fetch), expressed in hundredths of seconds.
- **CONNECT_TIME:** Specify the total elapsed time limit for a session, expressed in minutes.
- **IDLE_TIME:** Specify the permitted periods of continuous inactive time during a session, expressed in minutes. Long-running queries and other operations are not subject to this limit. When you set an idle timeout of X minutes, note that the session will take a couple of additional minutes to be terminated. On the client application side, the error message shows up only the next time, when the idle client attempts to issue a new command.
- **LOGICAL_READS_PER_SESSION:** Specify the permitted number of data blocks read in a session, including blocks read from memory and disk.
- **LOGICAL_READS_PER_CALL:** Specify the permitted number of data blocks read for a call to process a SQL statement (a parse, execute, or fetch).
- **PRIVATE_SGA:** Specify the amount of private space a session can allocate in the shared pool of the system global area (SGA).
- **COMPOSITE_LIMIT:** Specify the total resource cost for a session, expressed in service units. Oracle Database calculates the total service units as a weighted sum of CPU_PER_SESSION, CONNECT_TIME, LOGICAL_READS_PER_SESSION, and PRIVATE_SGA.
- Resource-related profile settings specified in the profile assigned to a user in a PDB get used without consulting resource-related settings in a profile assigned to the common user in the root. For example, if the profile local_prof that is assigned to user c##admin in a PDB has SESSIONS_PER_USER set to 2, then c##admin is only allowed only 2 concurrent sessions when he or she logs in to the PDB loc_prof is assigned to him, regardless of value of this setting in a profile assigned to him in the root.

Profile password parameters

- Profile password parameters in Oracle provide a way to enforce password policies for users assigned to a specific profile. These parameters are defined within the CREATE PROFILE statement and offer granular control over password behaviour within your database

- Use the following clauses to set password parameters.

Parameters that set lengths of time—that is, all the password parameters except `FAILED_LOGIN_ATTEMPTS` and `PASSWORD_REUSE_MAX`—are interpreted in number of days. For testing purposes you can specify minutes ($n/1440$) or even seconds ($n/86400$) for these parameters. You can also use a decimal value for this purpose (for example `.0833` for approximately one hour). The minimum value is 1 second. The maximum value is 24855 days. For `FAILED_LOGIN_ATTEMPTS` and `PASSWORD_REUSE_MAX`, you must specify an integer.

FAILED_LOGIN_ATTEMPTS: Specify the number of consecutive failed attempts to log in to the user account before the account is locked. If you omit this clause, then the default is 10 times.

PASSWORD_LIFE_TIME: Specify the number of days the same password can be used for authentication. If you also set a value for `PASSWORD_GRACE_TIME`, then the password expires if it is not changed within the grace period, and further connections are rejected. If you omit this clause, then the default is 180 days.

PASSWORD_REUSE_TIME and **PASSWORD_REUSE_MAX:** These two parameters must be set in conjunction with each other. `PASSWORD_REUSE_TIME` specifies the number of days before which a password cannot be reused. `PASSWORD_REUSE_MAX` specifies the number of password changes required before the current password can be reused. For these parameter to have any effect, you must specify a value for both of them.

- If you specify a value for both of these parameters, then the user cannot reuse a password until the password has been changed the number of times specified for `PASSWORD_REUSE_MAX` during the number of days specified for `PASSWORD_REUSE_TIME`.
- For example, if you specify `PASSWORD_REUSE_TIME` to 30 and `PASSWORD_REUSE_MAX` to 10, then the user can reuse the password after 30 days if the password has already been changed 10 times.

- If you specify a value for either of these parameters and specify UNLIMITED for the other, then the user can never reuse a password.
- If you specify DEFAULT for either parameter, then Oracle Database uses the value defined in the DEFAULT profile. By default, all parameters are set to UNLIMITED in the DEFAULT profile. If you have not changed the default setting of UNLIMITED in the DEFAULT profile, then the database treats the value for that parameter as UNLIMITED.
- If you set both of these parameters to UNLIMITED, then the database ignores both of them. This is the default if you omit both parameters.

PASSWORD_LOCK_TIME: Specify the number of days an account will be locked after the specified number of consecutive failed login attempts. If you omit this clause, then the default is 1 day.

PASSWORD_GRACE_TIME: Specify the number of days after the grace period begins during which a warning is issued and login is allowed. If you omit this clause, then the default is 7 days.

INACTIVE_ACCOUNT_TIME: Specify the permitted number of consecutive days of no logins to the user account, after which the account will be locked. The minimum value is 15 days. The maximum value is 24855. If you omit this clause, then the default is UNLIMITED.

PASSWORD_VERIFY_FUNCTION: You can pass a PL/SQL password complexity verification script as an argument to CREATE PROFILE by specifying PASSWORD_VERIFY_FUNCTION. Oracle Database provides a default script, but you can write your own function or use third-party software instead.

For function, specify the name of the password complexity verification routine. The function must exist in the SYS schema and you must have EXECUTE privilege on the function.

- Specify NULL to indicate that no password verification is performed.

- If you specify expr for any of the password parameters, then the expression can be of any form except scalar subquery expression.

Create Profile Command

- Use the CREATE PROFILE statement to create a profile, which is a set of limits on the database resources and the user password. Once you assign a profile to a user, then that user cannot exceed the database resource and password limits.
- The CREATE PROFILE statement allows you to create a new user profile.

Basic Syntax is :

CREATE PROFILE profile_name

LIMIT { resource_parameters | password_parameters};

In this syntax:

☞ First, specify the name of the profile that you want to create.

(profile_name)

☞ Second, specify the LIMIT on either database resources or password.

(resource_parameters and password_parameters)

- When you **create a user** without explicitly specifying a profile, Oracle will assign the **DEFAULT profile** to the user.
- To find the parameters of DEFAULT profile, you query the dba_profiles as shown in the following query:

```
SELECT * FROM dba_profiles WHERE PROFILE = 'DEFAULT'

ORDER BY resource_type, resource_name;
```

Examples:

Using Oracle CREATE PROFILE to set the resource limit

```
CREATE PROFILE CRM_USERS LIMIT
```

```

SESSIONS_PER_USER  UNLIMITED

CPU_PER_SESSION    UNLIMITED

CPU_PER_CALL       3000

CONNECT_TIME       15;

```

Now we can create user called crm with above profile like this:

```

CREATE USER crm IDENTIFIED BY abcd1234

PROFILE crm_users;

```

Using Oracle CREATE PROFILE to set the password

```

CREATE PROFILE erp_users LIMIT

FAILED_LOGIN_ATTEMPTS 5

PASSWORD_LIFE_TIME 90;

```

Now we can create a user named sap and set its profile to erp_users:

```

CREATE USER sap IDENTIFIED BY abcd1234 PROFILE erp_users;

```

The sap user is subject to the following password limits:

- ☞ The number of consecutive failed login attempts (FAILED_LOGIN_ATTEMPTS) is 5 before the account is locked.
- ☞ The number of days to change the password is 90 days.

Creating a profile with both resource limits and password parameters (power_user_profile):

```

CREATE PROFILE power_user_profile LIMIT SESSIONS_PER_USER 5
CPU_PER_SESSION 3600 PASSWORD_LIFE_TIME 365
PASSWORD_REUSE_TIME 14;

```

Manage Passwords with Profiles

- Oracle offers user profiles to set password management policies for database users., They define parameters that control password behaviour. We Use the CREATE PROFILE statement to define password parameters. Several profile parameters influence password management. Here are some common ones:
 - PASSWORD_LIFE_TIME: Defines the validity period of a password before it needs to be changed (in days).
 - PASSWORD_GRACE_TIME: Allows a grace period to change the password before the account locks (in days).
 - FAILED_LOGIN_ATTEMPTS: Sets the maximum failed login attempts before the account locks.

Control Resource Usage with Profiles

Oracle 19c utilizes profile resource parameter to manage and control resource consumption for users and sessions within the database. We Use the CREATE PROFILE statement to define resource limits. Oracle offers various resource types you can control with profiles. Some common ones include:

- CPU_PER_SESSION: Limits CPU usage per user session.
- CONNECT_TIME: Restricts the total connection time for a session.
- LOGICAL_READS_PER_SESSION: Limits the number of logical reads a session can perform.

Maintain Profiles

- Maintaining profiles in Oracle 19c involves a combination of creating, altering, and monitoring them to ensure effective resource management and password policies.
- Use the CREATE PROFILE statement to define profiles for password management or resource control. Specify relevant parameters based on your needs (e.g., password lifetime, CPU limits).
- Use the ALTER PROFILE statement to add, modify, or remove a resource limit or password management parameter in a profile.
- Example:

```
ALTER PROFILE new_profile

LIMIT PASSWORD_REUSE_TIME 90

PASSWORD_REUSE_MAX UNLIMITED;
```

- Changes made to a profile with an ALTER PROFILE statement affect users only in their subsequent sessions, not in their current sessions.
- Use the DROP PROFILE statement to remove a profile from the database. You can drop any profile except the DEFAULT profile.
 - Specify CASCADE to deassign the profile from any users to whom it is assigned. Oracle Database automatically assigns the DEFAULT profile to such users. You must specify this clause to drop a profile that is currently assigned to users.

Example:

```
DROP PROFILE app_user CASCADE;
```

- The DBA_PROFILES view provides information on all the profiles and the resource limits for each profile. The RESOURCE_COST view shows the unit cost associated with each resource. Each user can find information on his resources and limits in the USER_RESOURCE_LIMITS view.
- Profiles can be associated with users while executing CREATE USER or ALTER USER statements,

Overview of Database Security

- Oracle Database provides a rich set of default security features to manage user accounts, authentication, privileges, application security, encryption, network traffic, and auditing.
- When you create **user accounts**, you can secure them in a variety of ways. You can also create password profiles to better secure password policies
- Oracle Database provides several ways to configure **authentication** for users and database administrators. For example, you can authenticate users on the database level, from the operating system, and on the network
- We can use **privileges and roles** to restrict user access to data.
- A security policy for application developers should encompass areas such as password management and securing external procedures and application privileges.

- An **application context** is a name-value pair that holds the session information. You can retrieve session information about a user, such as the user name or terminal, and restrict database and application access for that user based on this information
- Oracle **Virtual Private Database (VPD)** enables you to filter users who access data.
- **Transparent sensitive data protection** enables you to find all table columns in a database that hold sensitive data. It is a way to find and classify table columns that hold sensitive information.
- We can use the **DBMS_CRYPTO PL/SQL** package to manually encrypt data. The DBMS_CRYPTO package provides four functions that enable you to perform asymmetric key operations for encryption, decryption, signing, and verification.
- Oracle Database **native network encryption and strong authentication** enables thin Java Database Connectivity (JDBC) clients to securely connect to Oracle databases
- We can configure your databases to use **strong authentication** with Oracle authentication adapters that support various third-party authentication services, including SSL with digital certificates
- We can **audit database** activities in general terms, such as auditing all SQL statements, SQL privileges, schema objects, and network activity.

Overview of Database Auditing

- Privileged users can create policies that track the changes that all users, including other privileged users, make in the database.
- **Auditing** is the monitoring and recording of database activity, from database users.
- We typically use auditing to monitor user activity. Auditing can be used to accomplish the following:
 - ☞ **Enable accountability for actions.** These include actions taken in a particular schema, table, or row, or affecting specific content.
 - ☞ **Deter users (or others, such as intruders) from inappropriate actions based on their accountability.**
 - ☞ **Investigate suspicious activity.** For example, if a user is deleting data from tables, then a security administrator can audit all connections to the database

and all successful and unsuccessful deletions of rows from all tables in the database.

- ☞ **Notify an auditor of the actions of an unauthorized user.** For example, an unauthorized user could be changing or deleting data, or the user has more privileges than expected, which can lead to reassessing user authorizations.
- ☞ **Support post-incident investigations.**
- ☞ **Monitor and gather data about specific database activities.** For example, the database administrator can gather statistics about which tables are being updated, how many logical I/Os are performed, or how many concurrent users connect at peak times.
- ☞ **Detect problems with an authorization or access control implementation.** For example, you can create audit policies that you expect will never generate an audit record because the data is protected in other ways. However, if these policies generate audit records, then you will know the other security controls are not properly implemented.
- ☞ **Address auditing requirements for compliance.** Regulations such as the following have common auditing-related requirements:
 - Sarbanes-Oxley Act
 - Health Insurance Portability and Accountability Act (HIPAA)
 - International Convergence of Capital Measurement and Capital Standards: a Revised Framework (Basel II)
 - Japan Privacy Law
 - European Union Directive on Privacy and Electronic Communications

Best Practices for Auditing

You should follow best practices guidelines for auditing.

- **As a general rule, design your auditing strategy to collect the amount of information that you need to meet compliance requirements, but focus on activities that cause the greatest security concerns.** For example, auditing every table in the database is not practical, but auditing tables with columns that contain

sensitive data, such as salaries, is. With both unified and fine-grained auditing, there are mechanisms you can use to design audit policies that focus on specific activities to audit.

- **Periodically archive and purge the audit trail data.** You can use the DBMS_AUDIT_MGMT package to purge audit records in several different ways. You should regularly review the collected audit records and establish a system for collecting and retaining audit records based on your site's retention policies. In addition to DBMS_AUDIT_MGMT, Oracle Data Safe and Oracle Audit Vault and Database Firewall provide features that enable you manage the archiving and purging of audit trail data.

Data Security Compliance

- Data security compliance refers to the policies and procedures that regulate how businesses and government agencies maintain data secured, confidential, and protected from unauthorized access. This compliance is most commonly used for customer data, although it can also apply to employee, financial, and other data types.
- When a corporation manages, maintains, and transmits data following the required compliance standards, it is said to have “compliant data.”
- **Why is Compliance Important for Data Security?**
 - Data compliance is all about meeting a set of rules or standards. On the other hand, information security focuses on safeguarding the confidentiality, integrity, and availability of information and technology assets within a company.
 - Data compliance entails adhering to compliance laws or compliance standards governing data and information security

Common Data Security Regulations

- HIPAA (Health Insurance Portability and Accountability Act) :USA
- GDPR (General Data Protection Regulation): EU
- CCPA (California Consumer Privacy Act)
- PCI-DSS e (Payment Card Industry Security Standards Council)

Standard Auditing

- Standard Audit is the most comprehensive and sufficient audit base in the Oracle database. It allows auditing an action, action kind, object, privilege, access user .
- There are 2 stages related to using the Standard Audit.
 - Activate audit mode.
 - Define the items to be audited i.e. define what actions to be created audit trail (Create Audit Trail).
- To activate the database audit mode, you need to access the Oracle with an user having the SYSDBA privilege.

Audit Trail

- An audit trail represents the collection of audit records from the target database trail such as UNIFIED_AUDIT_TRAIL, which provides documentary evidence of the sequence of activities that happen.
- A database audit trail is the source of audit records showing what has happened in the target database. When audit data collection is enabled for the specified database audit trail in an audit trail resource, the audit records are copied from the database's audit trail into Oracle Data Safe in near-real time.

Unified Auditing

- In unified auditing, the unified audit trail captures audit information from a variety of sources. Unified auditing enables you to capture audit records from the following sources:
 - Audit records (including SYS audit records) from unified audit policies and AUDIT settings
 - Fine-grained audit records from the DBMS_FGA PL/SQL package
 - Oracle Database Real Application Security audit records
 - Oracle Recovery Manager audit records
 - Oracle Database Vault audit records
 - Oracle Label Security audit records

- Oracle Machine Learning for SQL records
- Oracle Data Pump
- Oracle SQL*Loader Direct Load
- Oracle XML DB HTTP and FTP protocol messages
- The unified audit trail, which resides in a read-only table in the AUDSYS schema in the SYSAUX tablespace, makes this information available in a uniform format in the UNIFIED_AUDIT_TRAIL data dictionary view, and is available in both single-instance and Oracle Database Real Application Clusters environments. In addition to the user SYS, users who have been granted the AUDIT_ADMIN and AUDIT_VIEWER roles can query these views. If your users only need to query the views but not create audit policies, then grant them the AUDIT_VIEWER role.
- When the database is writeable, audit records are written to the unified audit trail. If the database is not writable, then audit records are written to new format operating system files in the \$ORACLE_BASE/audit/\$ORACLE_SID directory.

Benefits of the Unified Audit Trail

The benefits of a unified audit trail are many. For example:

- After unified auditing is enabled, it does not depend on the initialization parameters that were used in previous releases.
- The audit records, including records from the SYS audit trail, for all the audited components of your Oracle Database installation are placed in one location and in one format, rather than your having to look in different places to find audit trails in varying formats. This consolidated view enables auditors to co-relate audit information from different components. For example, if an error occurred during an INSERT statement, standard auditing can indicate the error number and the SQL that was executed. Oracle Database Vault-specific information can indicate whether this error happened because of a command rule violation or realm violation. Note that there will be two audit records with a distinct AUDIT_TYPE. With this unification in place, SYS audit records appear with AUDIT_TYPE set to Standard Audit.

- The management and security of the audit trail is also improved by having it in single audit trail.
- Overall auditing performance is greatly improved. By default, the audit records are automatically written to an internal relational table in the AUDSYS schema.
- You can create named audit policies that enable you to audit the supported components, as well as SYS administrative users. Furthermore, you can build conditions and exclusions into your policies.
- If you are using an Oracle Audit Vault and Database Firewall environment, then the unified audit trail greatly facilitates the collection of audit data, because all of this data will come from one location.

Separation of Audit Responsibilities with The AUDIT_ADMIN And AUDIT_VIEWER Roles

- Oracle provides two roles for users who perform auditing: AUDIT_ADMIN and AUDIT_VIEWER.
- The privileges that these roles provide are as follows:
 - **AUDIT_ADMIN role:** This role enables you to create unified and fine-grained audit policies, use the AUDIT and NOAUDIT SQL statements, view audit data, and manage the audit trail administration. Grant this role only to trusted users.
 - **AUDIT_VIEWER role:** This role enables users to view and analyze audit data. It provides the EXECUTE privilege on the DBMS_AUDIT_UTIL PL/SQL package. The kind of user who needs this role is typically an external auditor.
- To change audit policies or modify the audit trail (including purging old audit data), you must be granted the AUDIT_ADMIN role. An auditor can view audit data after being granted the AUDIT_VIEWER role.

Configure the Audit Trail

- Unified auditing supports custom unified audit policies, predefined unified auditing policies, and fine-grained auditing.

Selecting an Auditing Type

- You can audit general activities (such as SQL statement actions), commonly used auditing activities, or fine-grained audit scenarios.

Auditing SQL Statements, Privileges, and Other General Activities

- You can audit many types of objects, from SQL statements to other Oracle Database components, such as Oracle Database Vault..
- In addition, you can create policies that use conditions. However, if you want to audit specific columns or use event handlers, you must use fine-grained auditing.
- The general steps for performing this type of auditing are as follows:
 1. In most cases, use the CREATE AUDIT POLICY statement to create an audit policy. If you must audit application context values, then use the AUDIT statement.
 2. If you are creating an audit policy, then use the AUDIT statement to enable it and optionally apply (or exclude) the audit settings to one or more users, including administrative users who log in with the SYSDBA administrative privilege (for example, the SYS user).
AUDIT also enables you to create an audit record upon an action's success, failure, or both.
 3. Query the UNIFIED_AUDIT_TRAIL view to find the generated audit records.
 4. Periodically archive and purge the contents of the audit trail.

Auditing Commonly Used Security-Relevant Activities

- Oracle Database provides a set default unified audit policies that you can choose from for commonly used security-relevant audits.
- The general steps for performing this type of auditing are as follows:

❖ Auditing Activities with the Predefined Unified Audit Policies

☞ Logon Failures Predefined Unified Audit Policy

- The ORA_LOGIN_LOGOUT unified audit policy tracks failed logons only, but not any other kinds of logons.

☞ **Secure Options Predefined Unified Audit Policy**

- The **ORA_SECURECONFIG** unified audit policy provides all the secure configuration audit options.

☞ **Oracle Database Parameter Changes Predefined Unified Audit Policy**

- The **ORA_DATABASE_PARAMETER** policy audits commonly used Oracle Database parameter settings.

☞ **User Account and Privilege Management Predefined Unified Audit Policy**

- The **ORA_ACCOUNT_MGMT** policy audits commonly used user account and privilege settings.

☞ **Center for Internet Security Recommendations Predefined Unified Audit Policy**

- The **ORA_CIS_RECOMMENDATIONS** policy performs audits that the Center for Internet Security (CIS) recommends.

☞ **Oracle Database Real Application Security Predefined Audit Policies**

- You can use predefined unified audit policies for Oracle Database Real Application Security events.
- The **ORA_RAS_POLICY_MGMT** predefined unified audit policy audits policies for all Oracle Real Application Security administrative actions on application users, roles, and policies
- The **ORA_RAS_SESSION_MGMT** predefined unified audit policy audits policies for all runtime Oracle Real Application Security session actions and namespace actions.

☞ **Oracle Database Vault Predefined Unified Audit Policy for DVSYS and LBACSYS Schema**

- The **ORA_DV_AUDPOL** predefined unified audit policy audits Oracle Database Vault DVSYS and LBACSYS schema objects.

☞ **Oracle Database Vault Predefined Unified Audit Policy for Default Realms and Command Rules**

- The **ORA_DV_AUDPOL2** predefined unified audit policy audits the Oracle Database Vault default realms and command rules.

❖ **Auditing Specific Activities with Fine-Grained Auditing**

- Fine-grained auditing enables you to create policies that define specific conditions that must take place for the audit to occur.
- You can use fine-grained auditing to audit the following types of actions:
 - Accessing a table between 9 p.m. and 6 a.m. or on Saturday and Sunday
 - Using an IP address from outside the corporate network
 - Selecting or updating a table column
 - Modifying a value in a table column
- Unified audit policies can perform most of the operations that fine-grained audit policies can perform, except for the following actions:
 - Auditing specific columns. You can audit specific relevant columns that hold sensitive information, such as salaries or Social Security numbers.
 - Using event handlers. For example, you can write a function that sends an email alert to a security administrator when an audited column that should not be changed at midnight is updated.
- Fine-grained auditing has the following advantages over unified auditing:
 - It enables you to perform row value-based auditing. For example, you can audit updates to a salary column when the updated value is higher than a specified threshold, but not otherwise.

- You can use fine-grained auditing event handlers to proactively notify administrators or other users of specific events.
 - During a bulk data processing operation using BULK COLLECT and FORALL in PL/SQL to execute the same DML statement repeatedly for different bind variables, fine-grained auditing can capture the repeatedly executed statement with the appropriate bind variable values.
- Fine-grained auditing records are stored in the AUDSYS schema.
- Oracle provides roles for privileges needed to create fine-grained audit policies and to view and analyze fine-grained audit policy data. The fine-grained audit privileges are as follows:
- To create fine-grained audit policies, you must be granted the AUDIT_ADMIN role or the EXECUTE privilege on the DBMS_FGA package.
 - To view and analyze fine-grained audit data, you must be granted the AUDIT_VIEWER role.
- You can create **fine-grained audit policies in the CDB root, application root, CDB PDBs, and application PDBs.**
- Note the following general **rules about fine-grained audit policies in a multitenant environment:**
- You cannot create fine-grained audit policies on SYS objects.
 - You cannot create fine-grained audit policies, either local or application common, for extended data link objects.
 - When you create a fine-grained audit policy in the CDB root, the policy cannot be applied to all PDBs. It only applies to objects within the CDB root. (In other words, there is no such thing as a common fine-grained audit policy for the CDB root.) If you want to create a fine-grained audit policy to audit a common object's access in all the PDBs, then you must explicitly create that policy in each PDB and then enable it on the common objects that is accessible in the PDB.
 - When you create a fine-grained audit policy in a PDB, it applies only to objects within the PDB.

- You can create application common fine-grained audit policies only if you are connected to the application root and only within the BEGIN/END block. If you are connected to the application root and create the fine-grained audit policy outside the BEGIN/END block, then the fine-grained audit policy is created in the application root.
- You cannot create application common fine-grained audit policies on local PDB objects.
- If the application common fine-grained audit policy has a handler, then this handler must be owned by either an application common user or a CDB common user.
- You can create an application fine-grained audit policy on local (PDB) objects and CDB common objects. Because the policy is local to its container, the object on which the policy is defined is audited only in the particular container where the policy is defined. For example, if you create a fine-grained audit policy in the hr_pdb PDB, the object for which you create this policy must exist in the hr_pdb PDB.
- You cannot create local fine-grained audit policies in an application PDB on object linked and extended data link objects. On metadata-linked objects are allowed in the fine-grained audit policy.
- Application root local policies are allowed for all application common objects.
- When you create a fine-grained audit policy as a common audit policy in an application root, it will be effective in each PDB that belongs to this application root. Therefore, any access to the application common object and CDB common object (on which the application common fine-grained audit policy is defined) from the application PDB is audited in the fine-grained audit trail in that application PDB.
- When you create scripts for application install, upgrade, patch, or uninstall operations, you can include SQL statements within the ALTER PLUGGABLE DATABASE app_name BEGIN

INSTALL and ALTER PLUGGABLE DATABASE app_name END INSTALL blocks to perform various operations. You can include fine-grained audit policy statements only within these blocks.

- You can only enable, disable, or drop application common fine-grained audit policies from the application root, and from within a ALTER PLUGGABLE DATABASE app_name BEGIN INSTALL and ALTER PLUGGABLE DATABASE app_name END INSTALL block in a script.

Audit Policy Data Dictionary Views

- ☞ Data dictionary and dynamic views can be used to find detailed auditing information.

View	Description
ALL_AUDIT_POLICIES	Displays information about all fine-grained audit policies
ALL_DEF_AUDIT_OPTS	Lists default object-auditing options that are to be applied when objects are created
AUDIT_UNIFIED_CONTEXTS	Describes application context values that have been configured to be captured in the audit trail
AUDIT_UNIFIED_ENABLED_POLICIES	Describes all unified audit policies that are enabled in the database
AUDIT_UNIFIED_POLICIES	Describes all unified audit policies created in the database
AUDIT_UNIFIED_POLICY_COMMENTS	Shows the description of each unified audit policy, if a description was entered for the unified audit policy using the COMMENT SQL statement
AUDITABLE_SYSTEM_ACTIONS	Maps the auditable system action numbers to the action names
CDB_UNIFIED_AUDIT_TRAIL	Similar to the UNIFIED_AUDIT_TRAIL view, displays the audit records but from all PDBs in a multitenant environment. This view is available

	only in the CDB root and must be queried from there.
DBA_AUDIT_POLICIES	Displays information about fine-grained audit policies
DBA_SA_AUDIT_OPTIONS	Describes audited Oracle Label Security events performed by users, and indicates if the user's action failed or succeeded
DBA_XS_AUDIT_TRAIL	Displays audit trail information related to Oracle Database Real Application Security
DV\$CONFIGURATION_AUDIT	Displays configuration changes made by Oracle Database Vault administrators
DV\$ENFORCEMENT_AUDIT	Displays user activities that are affected by Oracle Database Vault policies
SYSTEM_PRIVILEGE_MAP (table)	Describes privilege (auditing option) type codes. This table can be used to map privilege (auditing option) type numbers to type names.
USER_AUDIT_POLICIES	Displays information about all fine-grained audit policies on table and views owned by the current user
UNIFIED_AUDIT_TRAIL	Displays all audit records
V\$OPTION	You can query the PARAMETER column for Unified Auditing to find if unified auditing is enabled
V\$XML_AUDIT_TRAIL	Displays standard, fine-grained, SYS, and mandatory audit records written in XML format files.

Auditing Activities with Unified Audit Policies and the AUDIT Statement

- ☞ You can audit the several types of activities, using unified audit policies and the AUDIT SQL statement.
- ☞ The kinds of activities that you can audit are as follows:
 - User accounts (including administrative users who log in with the SYSDBA administrative privilege), roles, and privileges
 - Object actions, such as dropping a table or a running a procedure
 - Application context values

— Activities from Oracle Database Real Application Security, Oracle Recovery Manager, Oracle Data Mining, Oracle Data Pump, Oracle SQL*Loader direct path events, Oracle Database Vault, and Oracle Label Security

☞ To find system actions to audit, you can query the AUDITABLE_SYSTEM_ACTIONS system table. To accomplish this, depending on what you want to audit, use the following:

- **Unified audit policies:** A unified audit policy is a named group of audit settings that enable you to audit a particular aspect of user behavior in the database. To create the policy, you use the CREATE AUDIT
- **POLICY statement:** The policy can be as simple as auditing the activities of a single user or you can create complex audit policies that use conditions. You can have more than one audit policy in effect at a time in a database. An audit policy can contain both system-wide and object-specific audit options. Most of the auditing that you will do for general activities (including standard auditing) requires the use of audit policies.
- **AUDIT and NOAUDIT SQL statements:** The AUDIT and NOAUDIT SQL statements enable you to, respectively, enable and disable an audit policy. The AUDIT statement also lets you include or exclude specific users for the policy. The AUDIT and NOAUDIT statements also enable you to audit application context values.
- **For Oracle Recovery Manager, you do not create unified audit policies:** The UNIFIED_AUDIT_TRAIL view automatically captures commonly audited Recovery Manager events.

Best Practices for Creating Custom Unified Audit Policies

- You can enable multiple policies at a time in the database, but ideally, limit the number of enabled policies.
- The unified audit policy syntax is designed so that you can write one policy that covers all the audit settings that your database needs. A good practice is to group related options into a single policy instead of creating multiple small policies. This

enables you to manage the policies much easier. As an example, each predefined audit policies contains multiple audit settings within one unified audit policy.

- Limiting the number of enabled audit policies for a user session has the following benefits:
 - It reduces the logon overhead that is associated with loading the audit policy's details into the session's UGA memory. If the enabled policy count is less, then less time is spent in loading the policy information.
 - It reduces the session's UGA memory consumption, because a fewer number of policies are required to be cached in UGA memory.
 - It makes the internal audit check functionality more efficient, which determines whether to generate an audit record for its associated event.
 - If you have configured a unified audit policy for LOGON statements, then audit records for both direct logins as well as ALTER SESSION and SET CONTAINER statements are generated.

Syntax for Creating a Unified Audit Policy

To create a unified audit policy, you must use the CREATE AUDIT POLICY statement.

When you create a unified audit policy, Oracle Database stores it in a first class object that is owned by the SYS schema, not in the schema of the user who created the policy.

CREATE AUDIT POLICY policy_name

```

    {      {privilege_audit_clause      [action_audit_clause      ]
    [role_audit_clause ]}

        | { action_audit_clause  [role_audit_clause ] }

        | { role_audit_clause }

    }

    [WHEN      audit_condition      EVALUATE      PER
    {STATEMENT|SESSION|INSTANCE}]

    [ONLY TOPLEVEL]

    [CONTAINER = {CURRENT | ALL}];

```

Here,

privilege_audit_clause describes privilege-related audit options.

action_audit_clause and *standard_actions* describe object action-related audit options

component_actions enables you to create an audit policy for Oracle Label Security, Oracle Database Real Application Security, Oracle Database Vault, Oracle Data Pump, or Oracle SQL*Loader

role_audit_clause enables you to audit roles.

WHEN *audit_condition* EVALUATE PER enables you to specify a function to create a condition for the audit policy and the evaluation frequency. You must include the EVALUATE PER clause with the WHEN condition

ONLY TOPLEVEL allows users to audit only the top-level operations that are performed for the actions that were configured as part of this audit policy.

CONTAINER, allows users to audit only the top-level operations that were performed for the actions that were configured as part of this audit policy

Examples:

The following statement creates unified audit policy **table_pol**, which audits the system privileges **CREATE ANY TABLE** and **DROP ANY TABLE**:

```
CREATE AUDIT POLICY table_pol
PRIVILEGES CREATE ANY TABLE, DROP ANY TABLE;
```

The following statement verifies that table_pol now appears in the AUDIT_UNIFIED_POLICIES view:

```
SELECT * FROM audit_unified_policies
WHERE policy_name = 'TABLE_POL';
```

The following statement creates unified audit policy dml_pol, which audits DELETE, INSERT, and UPDATE actions on table hr.employees, and all auditable actions on table hr.departments:


```
CREATE AUDIT POLICY dml_pol
  ACTIONS DELETE on hr.employees,
         INSERT on hr.employees,
         UPDATE on hr.employees,
         ALL on hr.departments;
```

The following query displays the standard RDBMS system actions that can be audited for the database:

```
SELECT name FROM auditable_system_actions
  WHERE component = 'Standard'
 ORDER BY name;
```

NAME

ADMINISTER KEY MANAGEMENT

ALL

ALTER ASSEMBLY

ALTER AUDIT POLICY

ALTER CLUSTER

...

Specify Audit Options

- ✓ Through SQL plus, an Oracle DBA can use audit and noaudit statements to configure audit options for the database. The audit statement allows you to set audit options at three levels:

Level	Effect
Statement	Audits specific SQL statements or groups of statements that affect a particular type of database object. For example, AUDIT TABLE audits the CREATE TABLE, TRUNCATE TABLE, COMMENT ON TABLE, and DELETE [FROM] TABLE statements.
Privilege	Audits SQL statements that are executed under the umbrella of a specified system privilege. For Example, AUDIT CREATE ANY TRIGGER audits statements issued using the CREATE ANY TRIGGER system privilege.
Object	Audits specific statements on specific objects, such as ALTER TABLE on the employee table

- ✓ To use the audit statement to set statement and privilege auditing options a DBA must be assigned AUDIT SYSTEM privileges. To use the audit statement to set object audit options, the DBA must own the object to be audited or be assigned the AUDIT ANY privilege within Oracle. Privilege assignments are covered in the following section.
- ✓ Audit statements that set statement and privilege audit options can also include a BY clause to supply a list of specific users or application proxies to audit, and thus limit the scope of the statement and privilege audit options.

Statement Audit Options (User sessions)

The following statement audits user sessions of users Bill and Lori.

```
AUDIT SESSION BY scott, lori;
```

The statement AUDIT SESSION BY scott, lori; in Oracle 19c enables auditing for all successful and unsuccessful session connection attempts (logins and logouts) for the users scott and lori.

Here's a breakdown of the statement:

AUDIT SESSION: This clause specifies the type of audit option being enabled. In this case, it's session auditing.

BY scott, lori: This clause defines the target users for whom session auditing will be enabled. Here, it applies to both scott and lori.

Impact of the Statement:

Whenever scott or lori attempts to connect to the database (successful or failed), a record will be logged in the Oracle audit trail. This information includes details like:

- Username attempting the connection
- Timestamp of the attempt
- Terminal/source used for connection (if applicable)
- Success/failure of the attempt
- Additional details depending on the audit level configured

Privilege Audit Options

The following statement audits all successful and unsuccessful uses of the DELETE ANY TABLE system privilege:

AUDIT DELETE ANY TABLE BY ACCESS WHENEVER NOT SUCCESSFUL;

The statement **AUDIT DELETE ANY TABLE BY ACCESS WHENEVER NOT SUCCESSFUL;** audits all unsuccessful attempts to use the DELETE ANY TABLE system privilege in Oracle 19c. Here's a breakdown:

AUDIT: This keyword initiates the definition of an audit option.

DELETE ANY TABLE: This specifies the specific privilege to be audited. In this case, it's the DELETE ANY TABLE privilege, which allows deletion of rows from any table in the database.

BY ACCESS: This clause indicates that auditing will be recorded on a per-user basis. Each unsuccessful attempt will be tied to the specific user who tried to use the privilege.

WHENEVER NOT SUCCESSFUL: This clause defines the condition for audit record generation. Only attempts that fail to execute (due to insufficient privileges, syntax errors, etc.) will be logged.

What Gets Audited?

When a user attempts to use the DELETE ANY TABLE privilege and the attempt fails, the following information will likely be recorded in the audit trail:

- Username attempting the operation
- Timestamp of the attempt
- Statement or action that triggered the privilege usage
- Reason for failure (if applicable)
- Additional details depending on the configured audit level

Object Audit Options

The following statement audits all successful SELECT, INSERT, and DELETE statements on the dept table owned by user jward:

```
AUDIT SELECT, INSERT, DELETE ON jward.dept BY ACCESS WHENEVER SUCCESSFUL;
```

The statement `AUDIT SELECT, INSERT, DELETE ON jward.dept BY ACCESS WHENEVER SUCCESSFUL;` enables auditing for all successful attempts to perform SELECT, INSERT, and DELETE operations on the table named dept owned by the schema jward in Oracle 19c. Here's a breakdown:

AUDIT: This keyword initiates the definition of an audit option.

SELECT, INSERT, DELETE: These specify the specific Data Manipulation Language (DML) statements to be audited.

ON jward.dept: This clause defines the target table for auditing. It focuses on the dept table owned by the schema jward.

BY ACCESS: This indicates that auditing will be recorded on a per-user basis. Each successful DML operation will be tied to the specific user who performed it.

WHENEVER SUCCESSFUL: This clause defines the condition for audit record generation. Only successful executions of SELECT, INSERT, and DELETE statements on the jward.dept table will be logged.

What Gets Audited?

Whenever a user successfully performs a SELECT, INSERT, or DELETE operation on the jward.dept table, information will likely be recorded in the audit trail. This information might include:

- Username performing the operation
- Timestamp of the operation
- Type of DML statement executed (SELECT, INSERT, DELETE)
- Data involved in the operation (depending on the audit level configured)
- Additional details about the session and environment